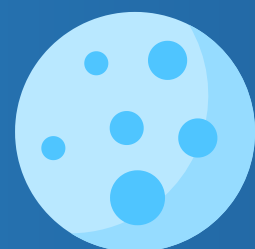




PANDEMIC

Projeto de Software
Parte 2

Equipe



Patricia Carvalho de Moraes
Matheus da Rocha Sena
Italo Ferreira da Silva
Fernando Rene Cabral Araujo
Mateus Sacramento de Matos
Lucas Araujo Sodre

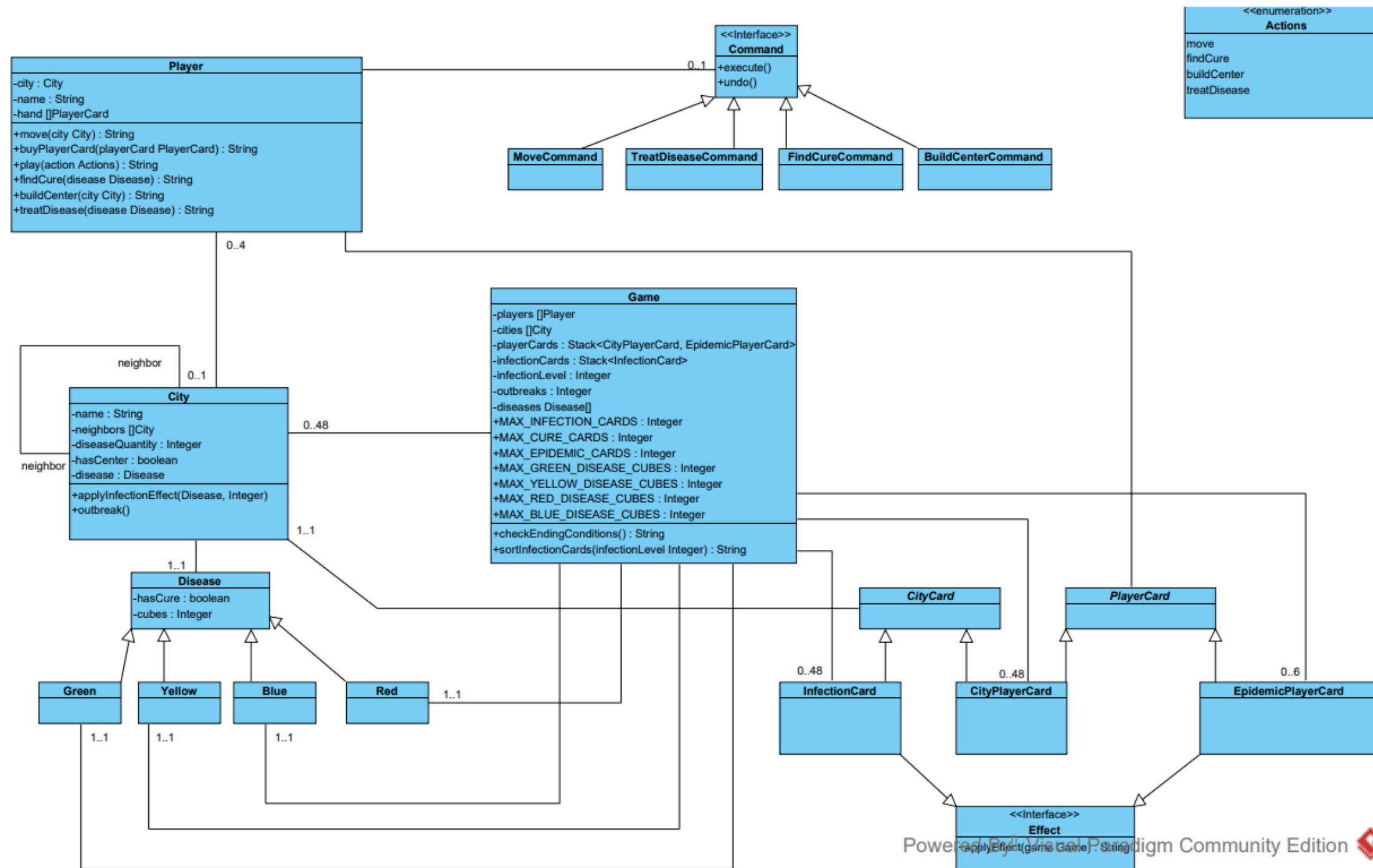


 <https://github.com/Rarkunho/pandemic-uff-gpms-ps.git>  



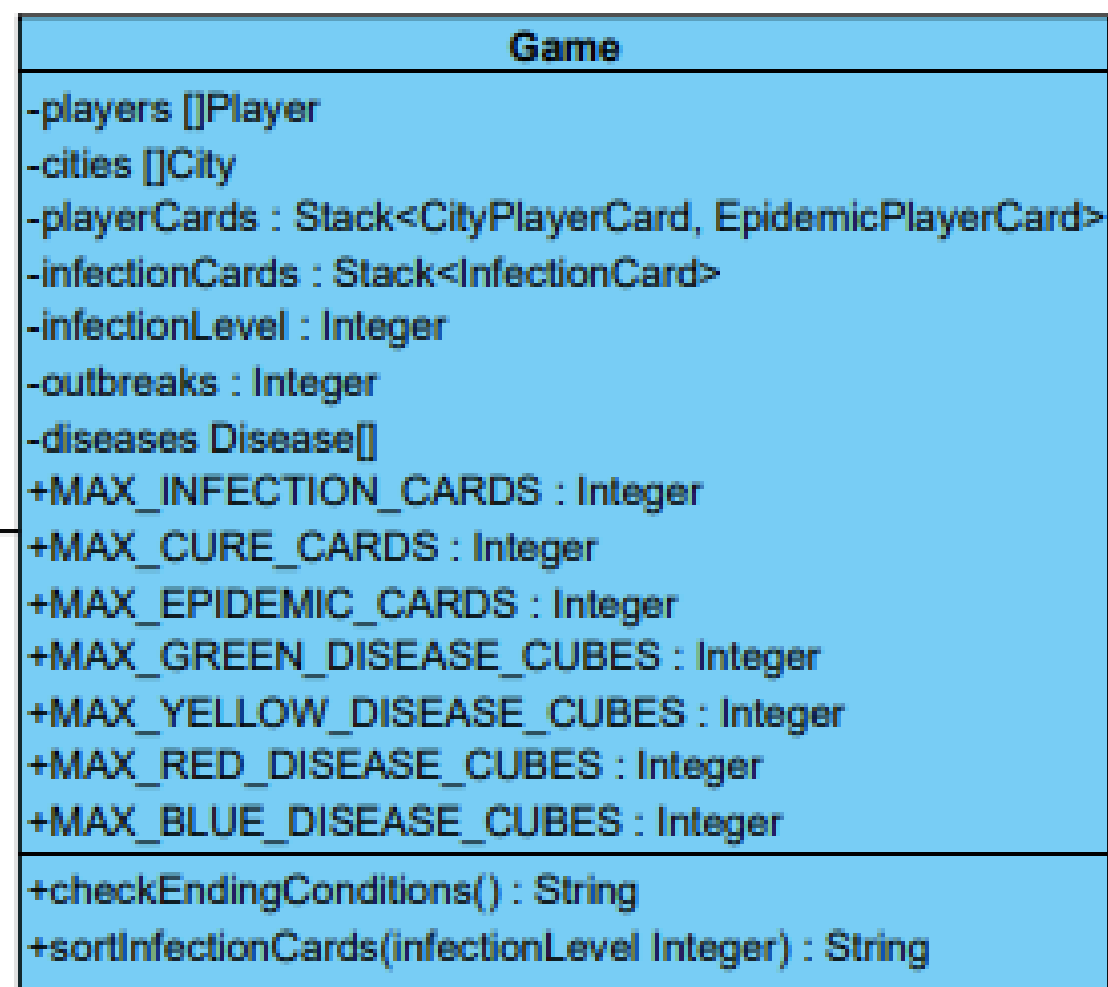


DIAGRAMA DE CLASSES





PADRÃO GRASP



GRASP: Controller (Controlador)

Atribui a responsabilidade de receber e coordenar eventos do sistema a uma classe que represente o sistema como um todo ou um cenário de caso de uso.

A classe *Game* atua como controlador central do sistema, gerencia o estado global do jogo e coordena as interações entre os principais componentes através de métodos como *checkEndingConditions* e *sortInfectionCards*



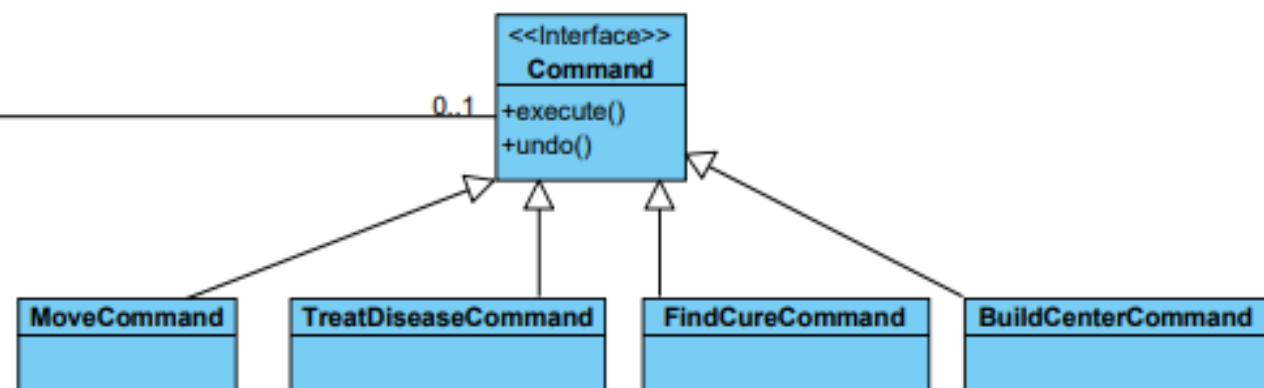
PADRÃO GRASP

GRASP: Baixo Acoplamento e Alta Coesão

O acoplamento mede o quão fortemente um elemento está conectado a outros elementos, enquanto a coesão é uma medida de quão relacionadas e focadas são as responsabilidades de um elemento.



o uso de interfaces como *Command* e *Effect* promove baixo acoplamento entre quem invoca uma ação e quem a executa. As classes têm responsabilidades bem definidas (alta coesão), como *Player*, focado nas ações do jogador, e *Disease*, focada nas características da doença



Powered By Visual Paradigm Community Edition



PADRÃO GOF

GoF: Command

Encapsula uma solicitação como um objeto, permitindo parametrizar clientes com diferentes solicitações, enfileirar ou registrar solicitações e suportar operações que podem ser desfeitas.



O diagrama mostra a interface *Command* com os métodos *execute()* e *undo()*, além das implementações concretas, como *MoveCommand* e *TreatDiseaseCommand*, permitindo que as ações do jogo sejam encapsuladas como objetos.

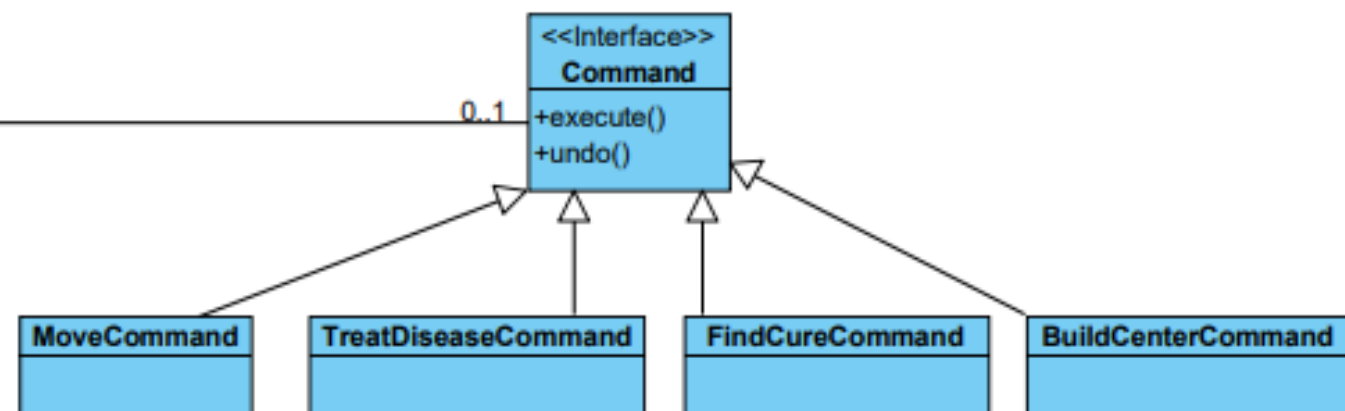
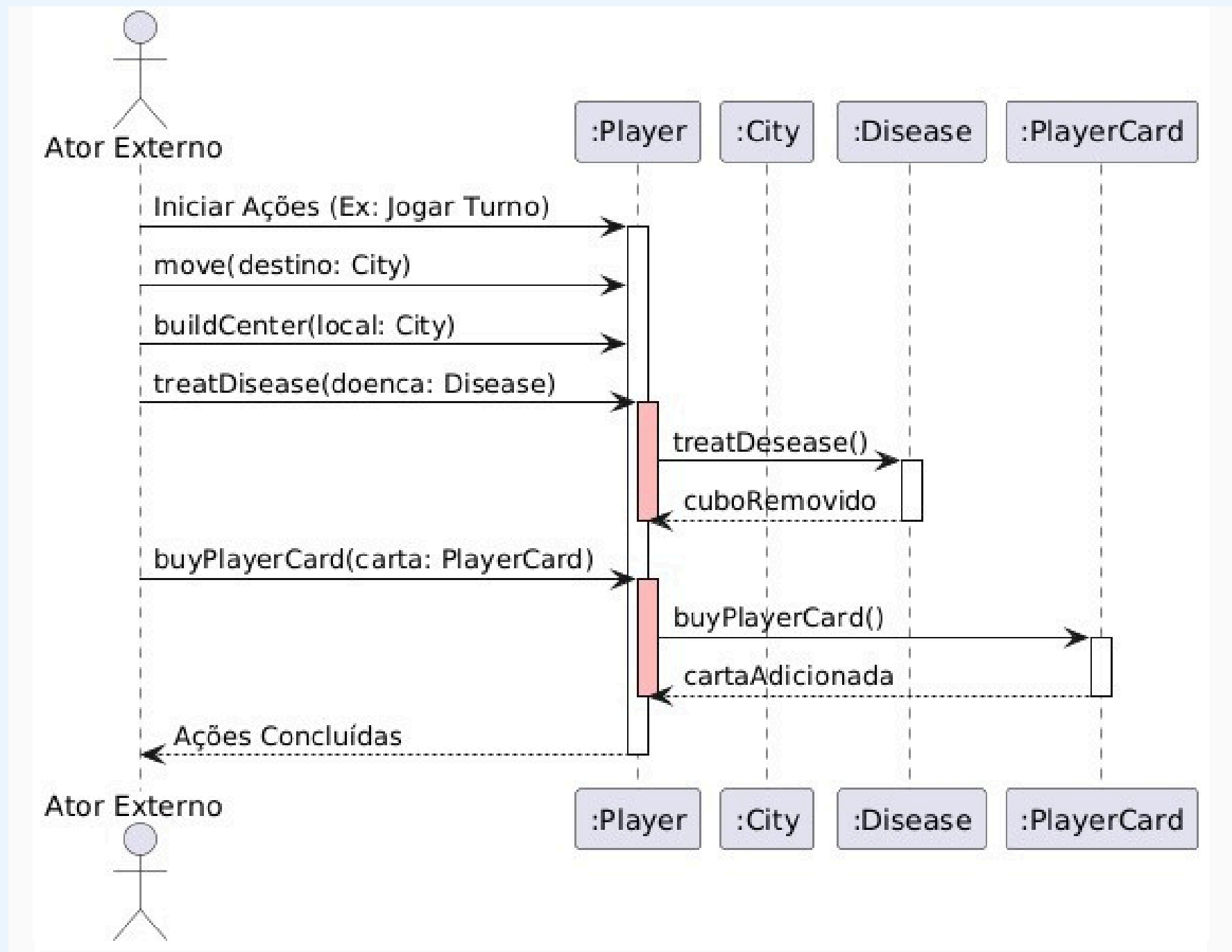


DIAGRAMA DE SEQUÊNCIA

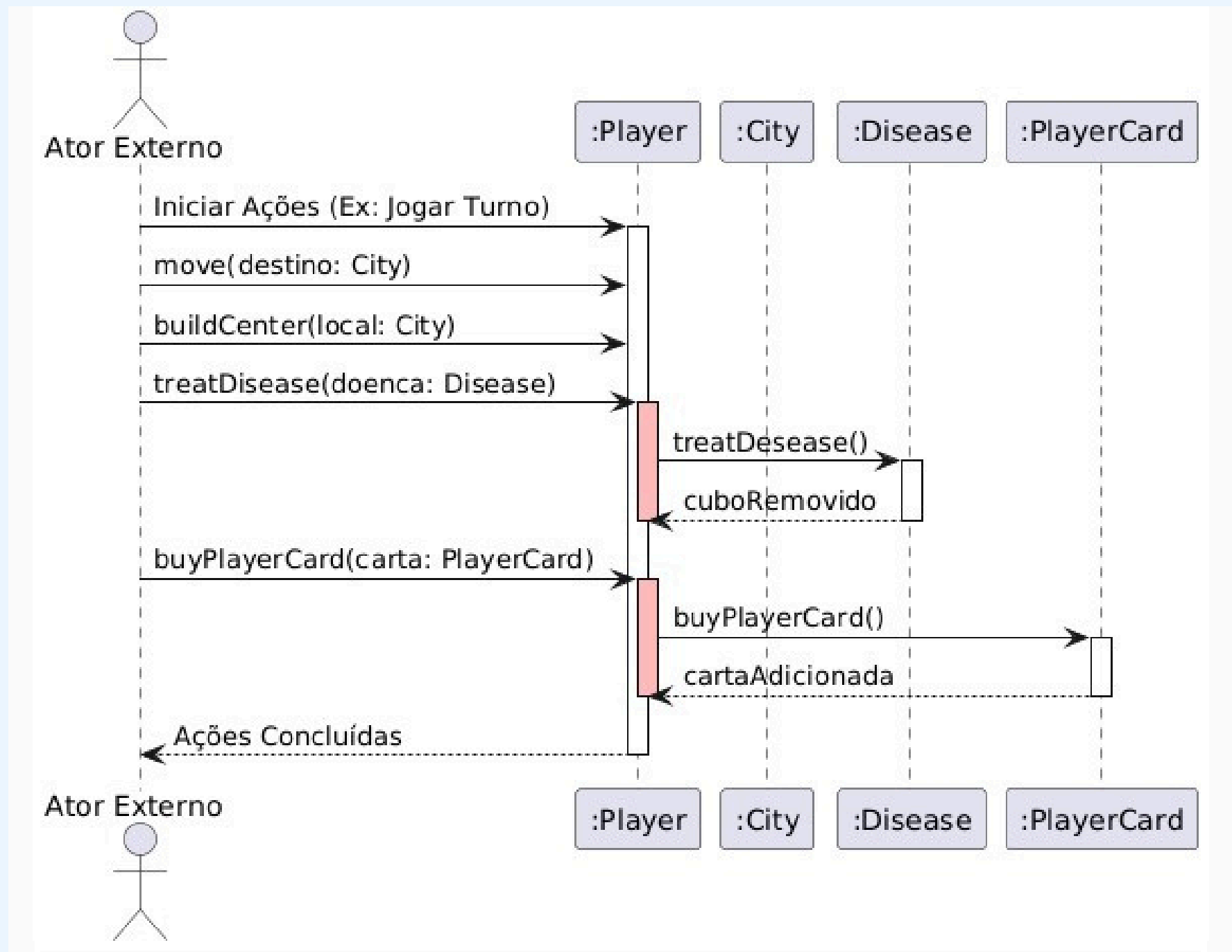


1. Início das Ações: envia uma mensagem para o objeto *Player*, que indica o começo de um conjunto de ações que o jogador pode realizar em seu turno.

2. move(destino: City): envia uma mensagem *move* para o objeto *Player*, indicando que o jogador deseja se mover para uma *City* específica

3. buildCenter(local: City): envia uma mensagem *buildCenter* para o objeto *Player*, com o parâmetro *local* sendo uma *City*

DIAGRAMA DE SEQUÊNCIA



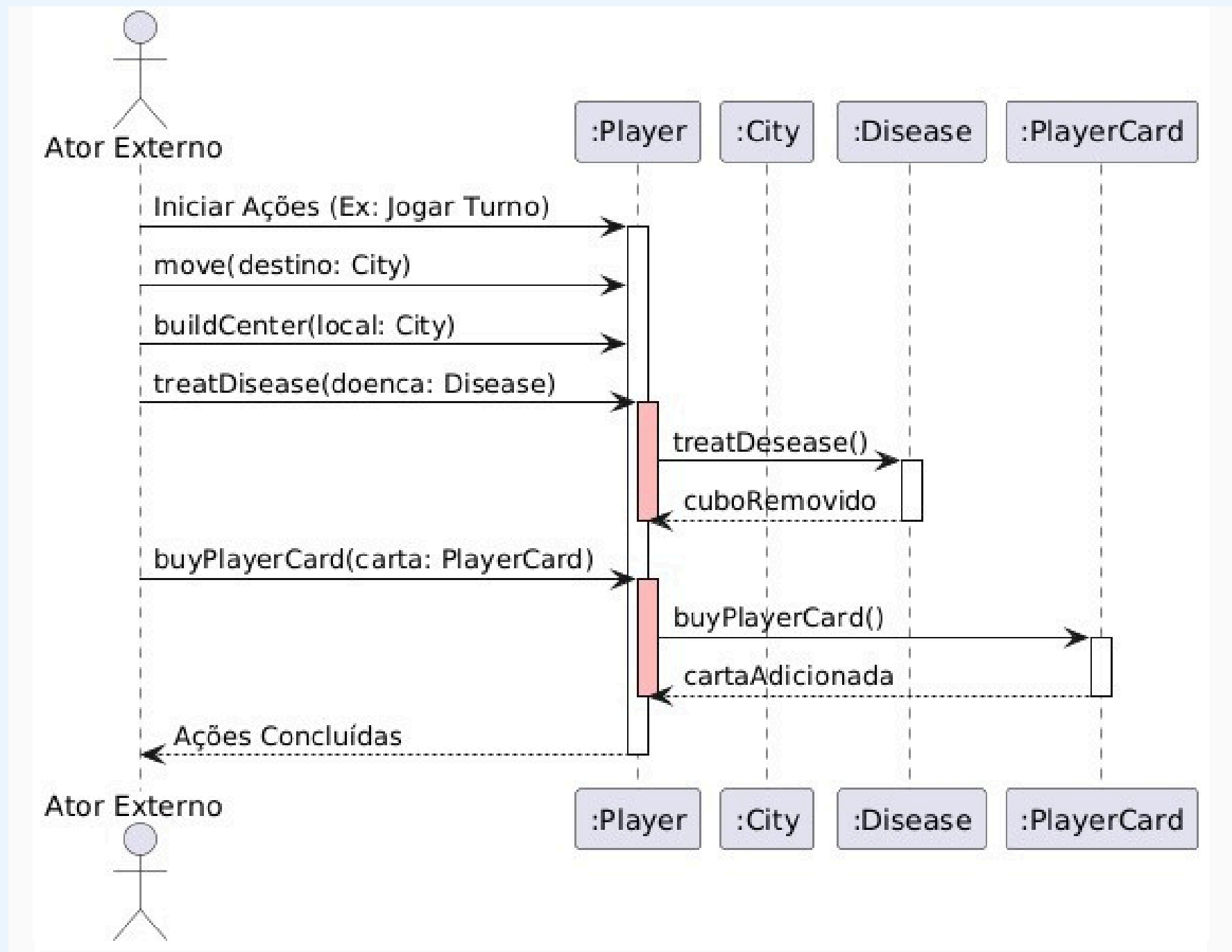
4. **treatDisease(doenca: Disease):**

envia a mensagem *treatDisease* para o objeto *Player*, passando uma *Disease* como parâmetro. Ao receber a mensagem *treatDisease*, o objeto *Player* envia uma mensagem *treatDisease()* para o objeto *Disease*, para processar o tratamento.

* Retorno *cuboRemovido*: O objeto *Disease* responde ao *Player* que um cubo foi removido



DIAGRAMA DE SEQUÊNCIA



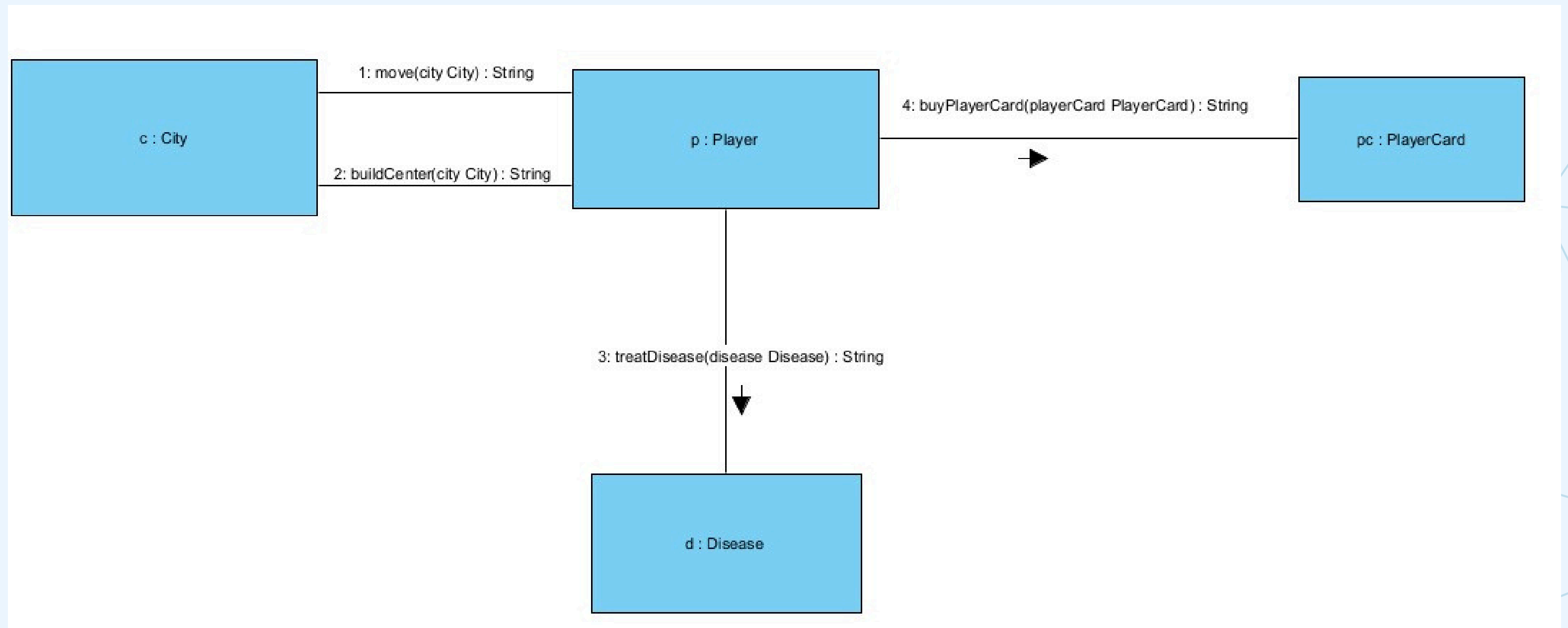
5. buyPlayerCard(carta: PlayerCard):

mensagem *buyPlayerCard* para o objeto *Player*, com o parâmetro carta, Isso representa a ação do jogador de tentar adquirir uma carta.

* Retorno *cartaAdicionada*: O objeto *PlayerCard* responde ao *Player* com uma mensagem de retorno *cartaAdicionada*. Isso confirma que a carta foi adicionada com sucesso ao jogador.

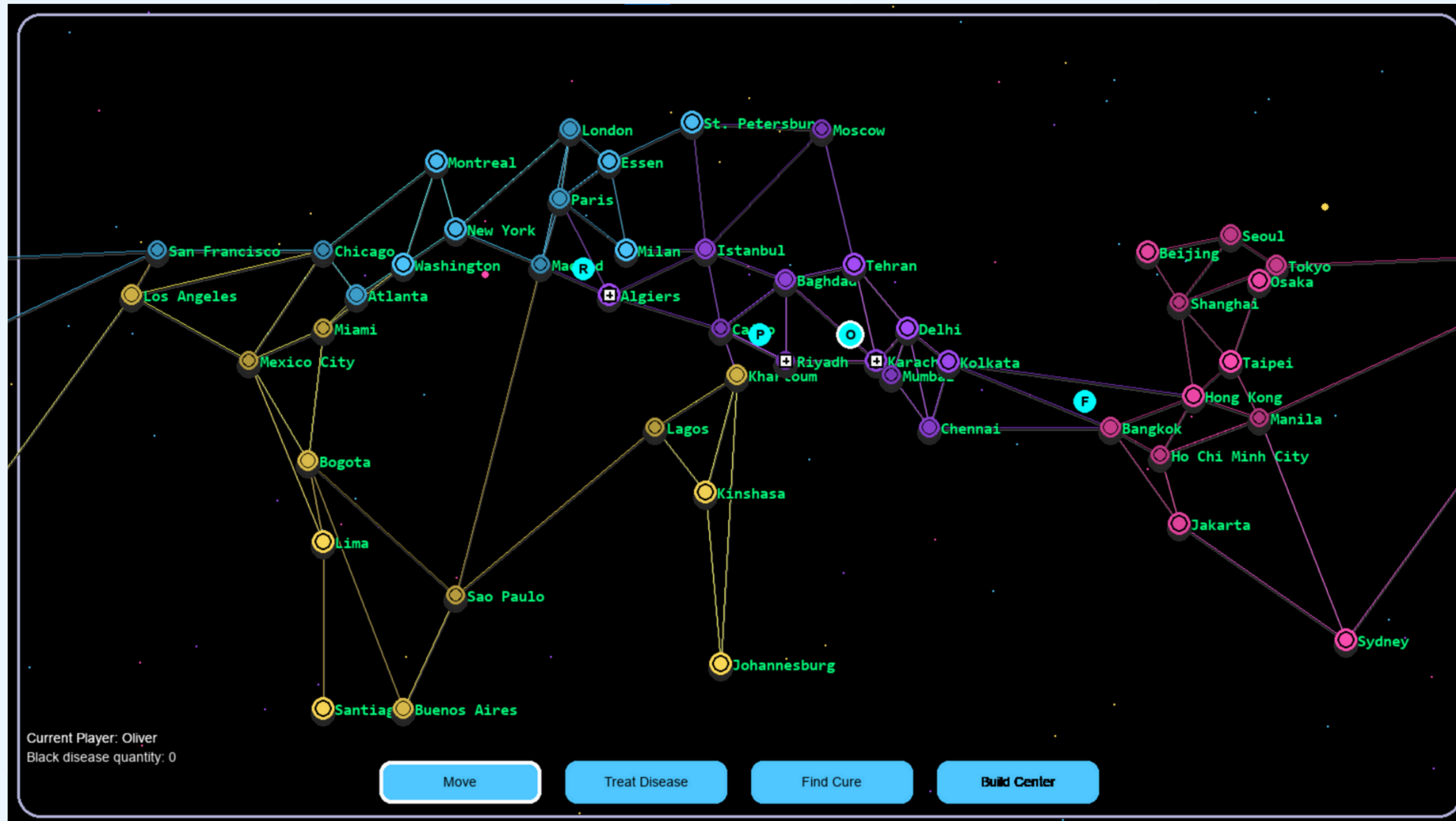


DIAGRAMA DE SEQUÊNCIA

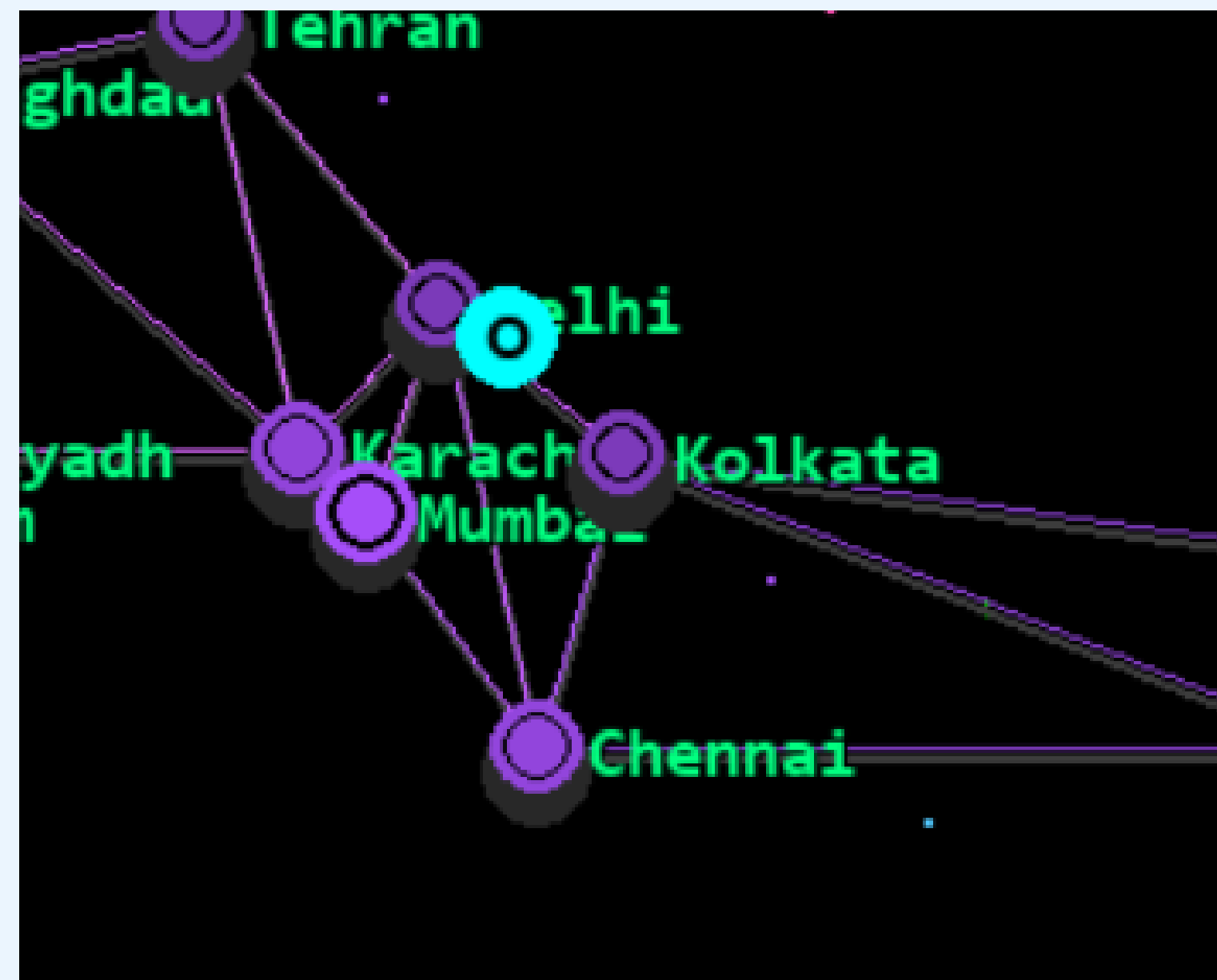
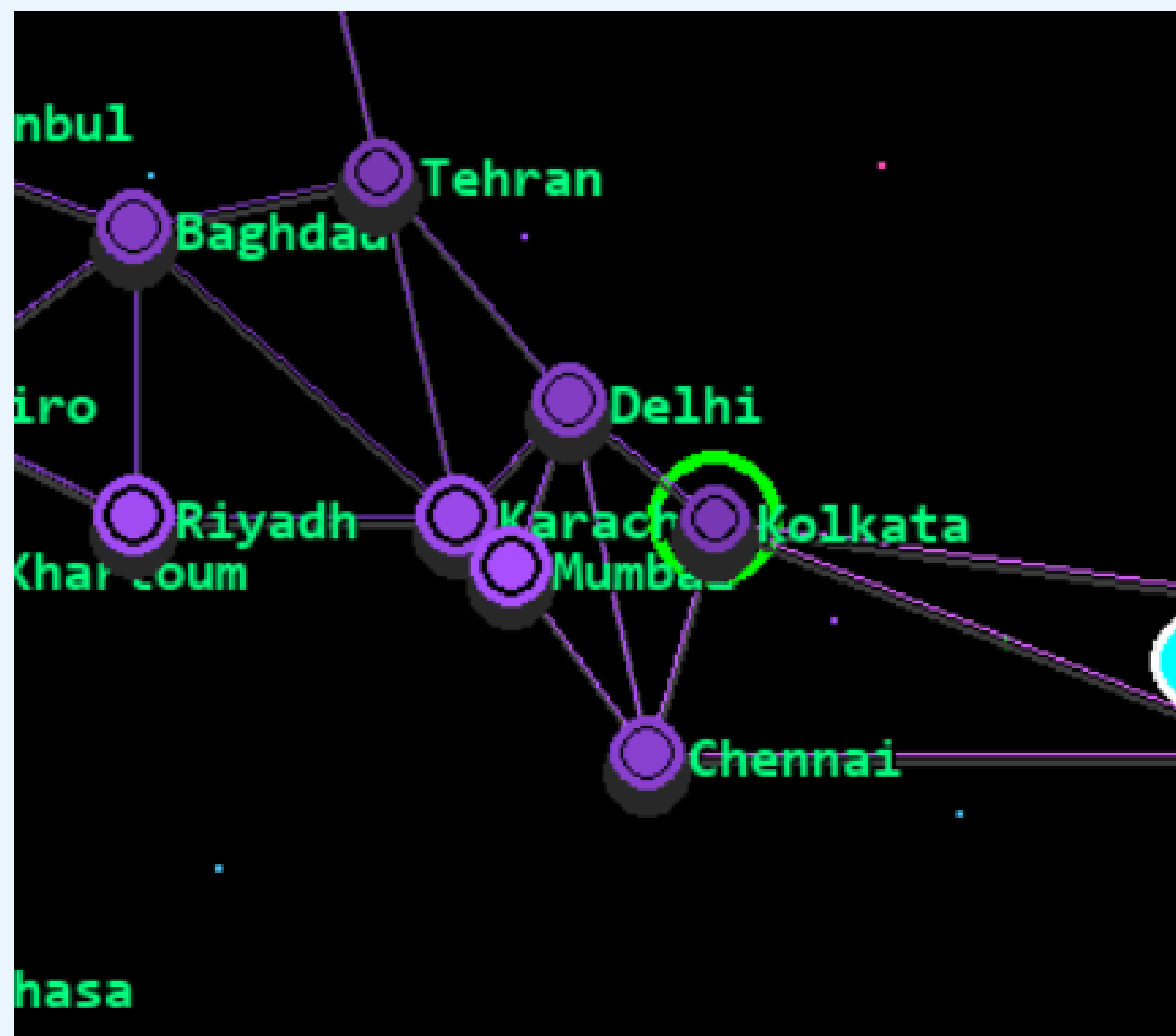




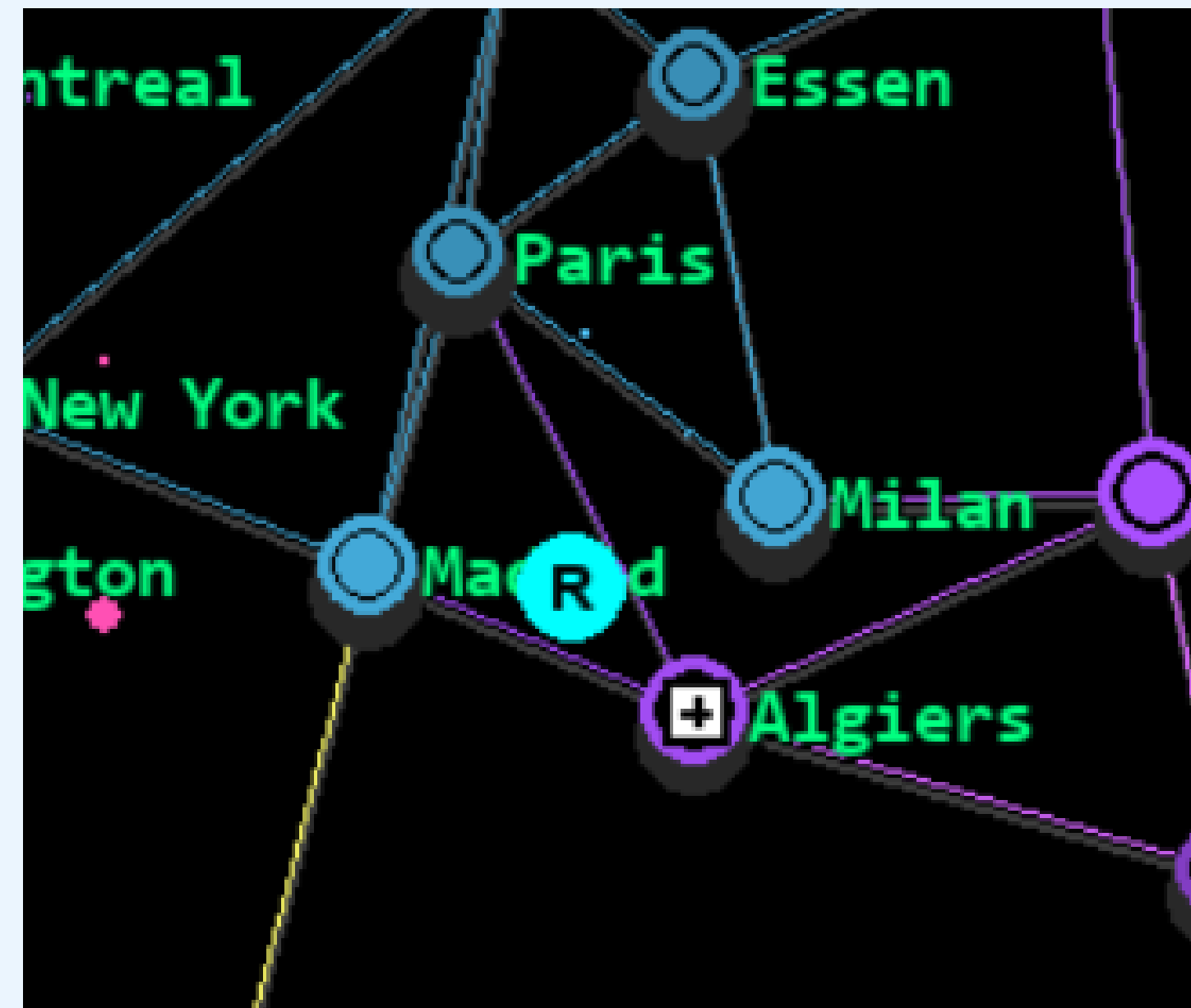
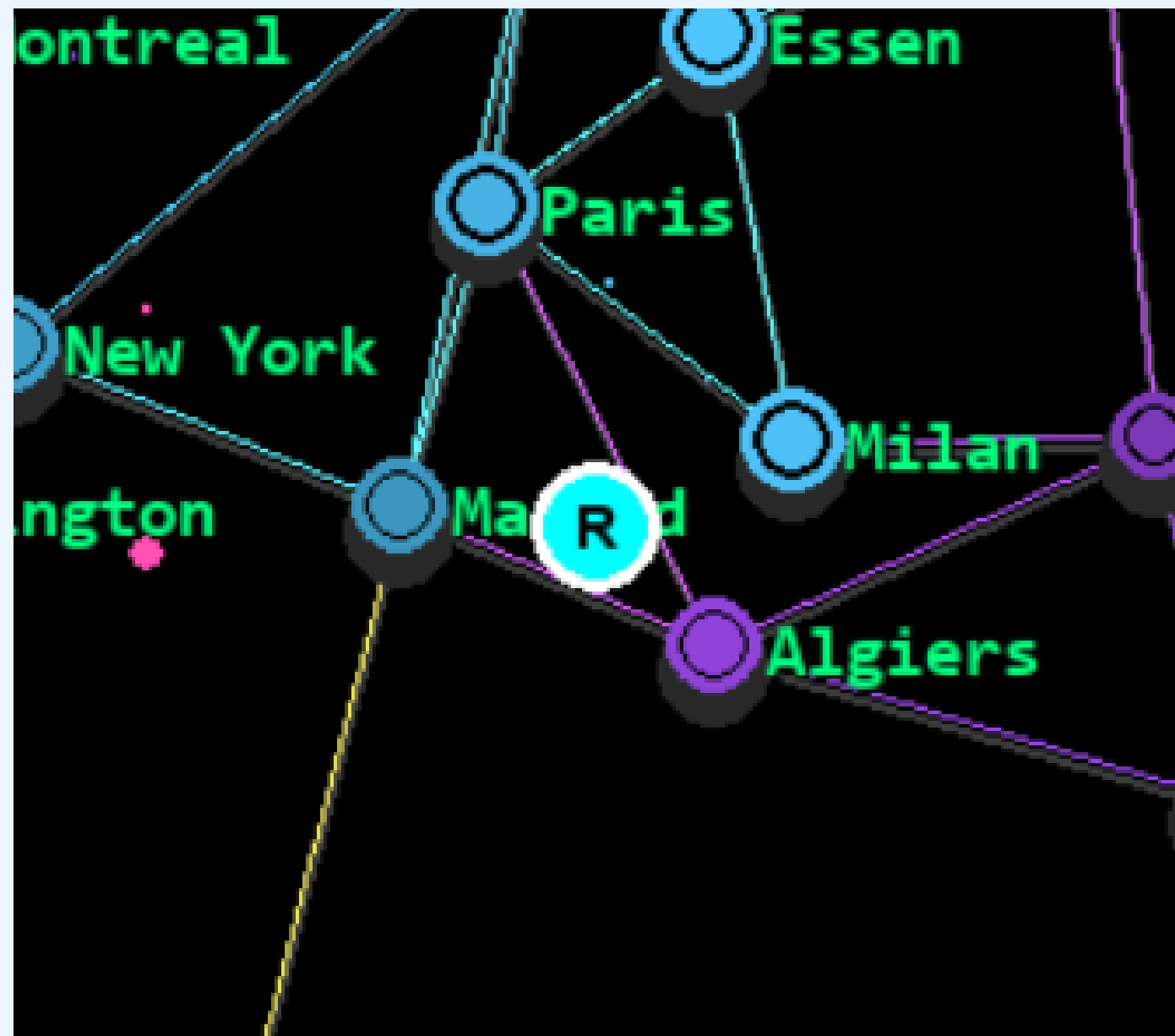
Demo



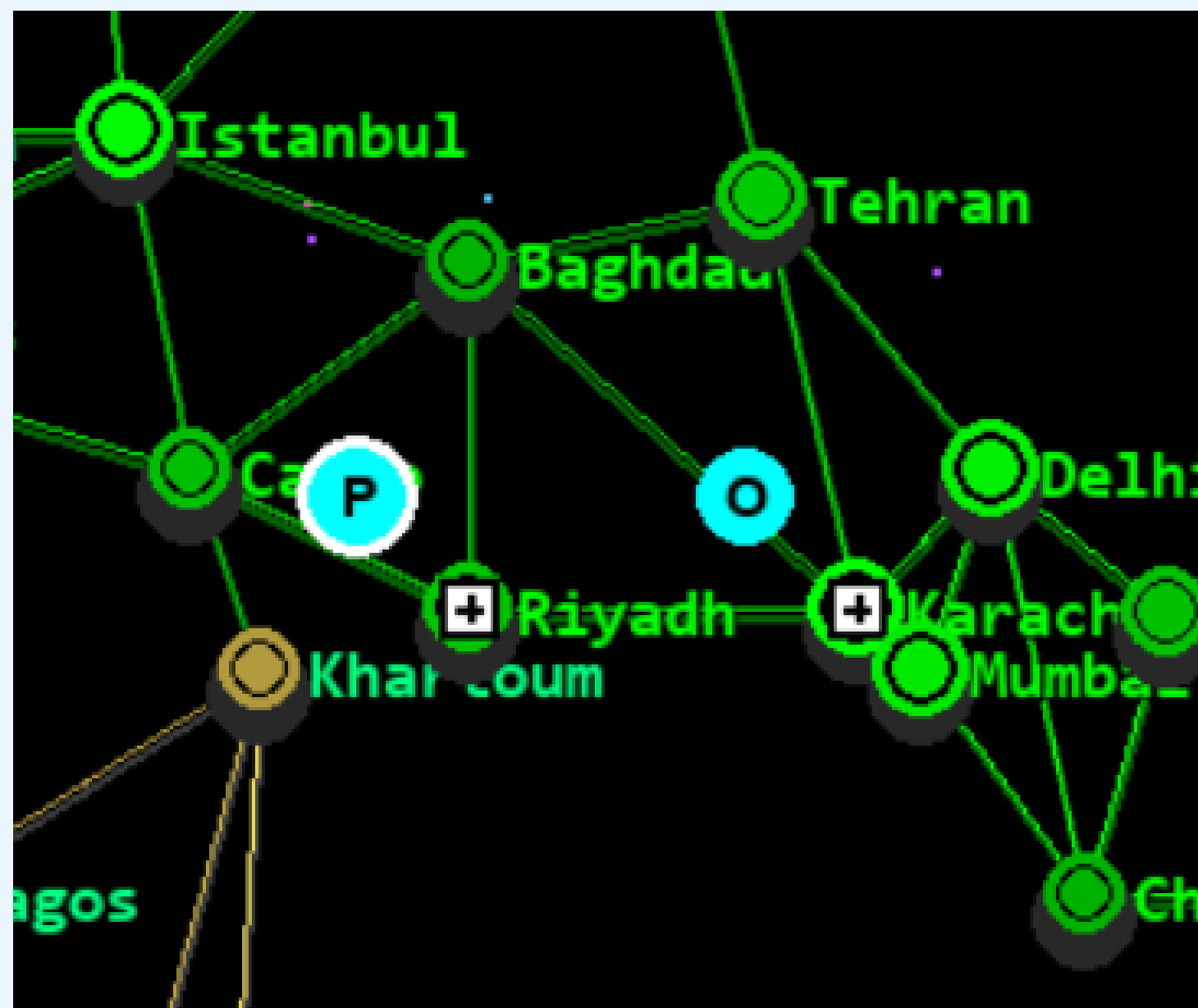
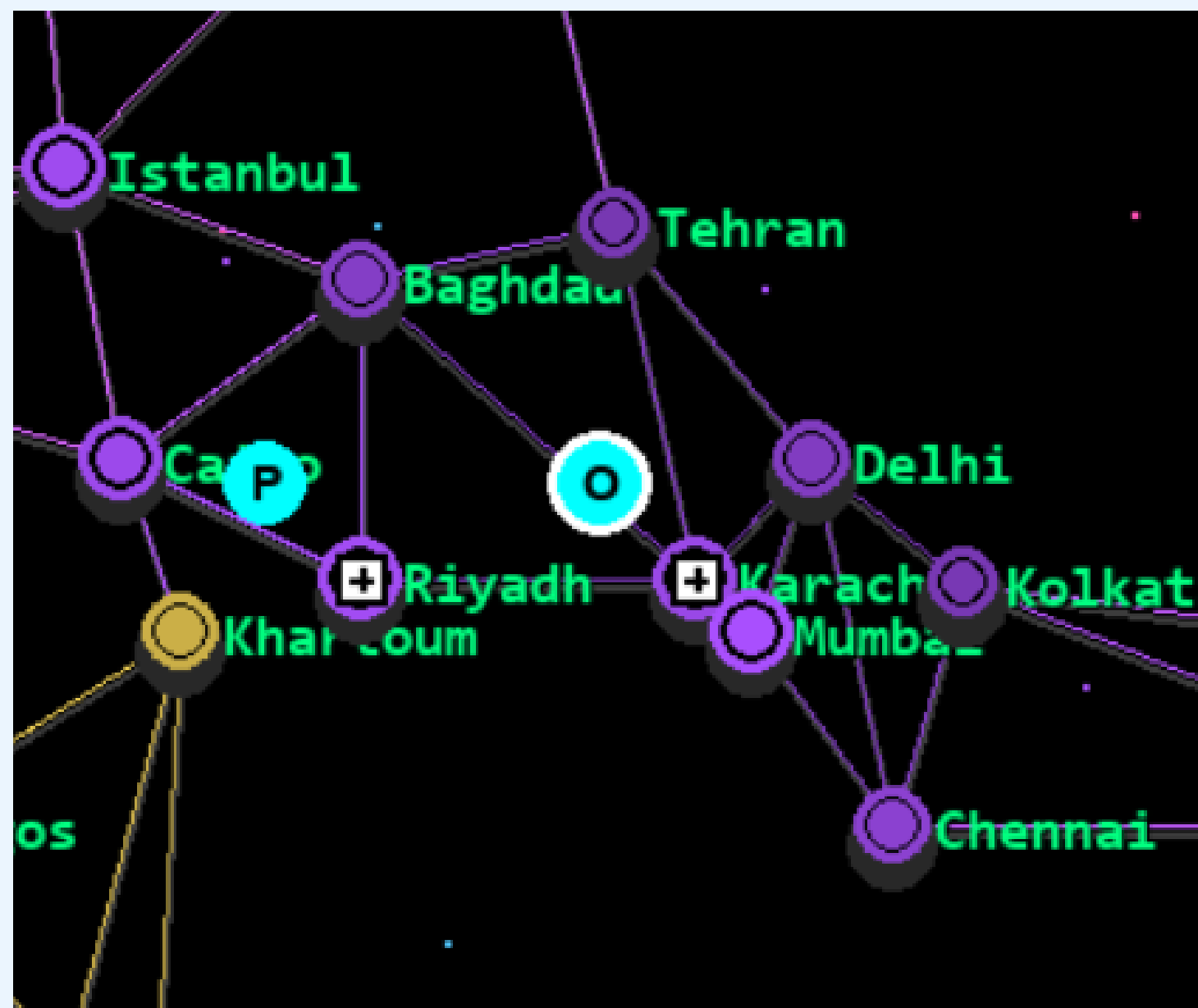
Demo - Movimentação



Demo - Construire Centro



Demo - Encontrar Cura



The background is a deep blue with abstract, flowing shapes in shades of teal and purple. In the center, there is a faint, semi-transparent image of a person wearing a VR headset, with the text 'VRQUEST' and 'VR' visible. Below this, a car is also faintly visible. The overall aesthetic is futuristic and tech-oriented.

OBRIGADO!